

FSA v5 port reproduction manual

Step-by-step recipe with per-phase verification gates
From the FSA v5 technical guide and Lean4 source
to a verified closed-loop SMC²-MPC bench

Ajay Talati + Claude

2026-05-09

Abstract

This document is the operational companion to the *LEAN4-First Charter*, the *FSA v5 Technical Guide*, and the *LaTeX* $\rightarrow$ *Lean4* $\rightarrow$ *Julia pipeline* doc. Given two on-disk inputs — the FSA v5 technical guide (`LaTeX_docs/FSA_version_5_technical_guide.tex`) and the Lean4 source files at `Fsa/V5/` — this manual walks a fresh human or AI agent through the seven phases needed to produce a verified closed-loop SMC²-MPC bench. Each phase is a small set of files, an imperative procedure, and a single-command verification gate. The final gate is an end-to-end T=7 day bench that should complete in ~ 6 minutes on RTX 5090 and produce `data.jld2`, three diagnostic PNGs, and a recovered final state matching the verified reference run on 2026-05-09.

The manual is deliberately operational (*do X; verify Y; then do Z*); the chronological narrative of how the port was constructed is captured separately in `models/fsa_v5/PORT_LOG.md`.

Contents

1	Goal and overview	3
1.1	One-sentence goal	3
1.2	Why this manual exists	3
1.3	Where this manual sits in the doc set	3
1.4	The seven port phases	3
1.5	How to read this manual	3
2	Prerequisites	4
3	Phase 0 — Inputs	4
4	Phase 1 — Build the v5 Lean4 CLI	4
5	Phase 2 — Write the Julia CPU port	6
6	Phase 3 — Differential test	7
6.1	Background: what is a differential test, and why use one here?	7
6.2	Implementation in this manual	9
7	Phase 4 — GPU port (SOFT cost only)	9
8	Phase 5 — Bench driver	11
9	Phase 6 — Plotters, production gating, and launchers	12

10 Verification gates — final checklist	13
11 Troubleshooting and known gotchas	13
A Complete file tree (post-port)	14
B Command index	15
C Differential-test coverage table	16

1 Goal and overview

1.1 One-sentence goal

Given the FSA v5 tech guide and the Lean4 source files in `Fsa/V5/`, produce a verified Julia port that runs the closed-loop SMC²-MPC bench end-to-end.

1.2 Why this manual exists

The work to date is captured only in conversational history and in a chronological `PORT_LOG.md`. Neither is a stable artefact a future reader can pick up and execute without reverse-engineering the order of operations and the rationale at each step.

LLM coding agents are a lossy medium for code: the LEAN4-First Charter exists precisely because pattern-matching agents introduce transcription errors when porting math. The differential test catches transcription errors at machine precision, but only if a future port is structured so that the diff-test gates apply. This manual prescribes the structure, the gates, and the per-phase pass criteria so that the verifiability of the work is preserved.

The result of every closed-loop bench — recovered θ posteriors, controller plans, A -accumulation traces — is only as trustworthy as the pipeline that produced it. The manual is the pipeline’s contract: *if you follow these steps and hit these gates, you have the same pipeline that produced the verified reference run on 2026-05-09.*

1.3 Where this manual sits in the doc set

Four LaTeX documents in `version_1_5_LEAN/LaTeX_docs/` answer four orthogonal questions (Figure 1): **why** we use a Lean4-first discipline, **what** the v5 model actually is, **how** the pipeline is architected, and (this manual) **how** to execute the port operationally.

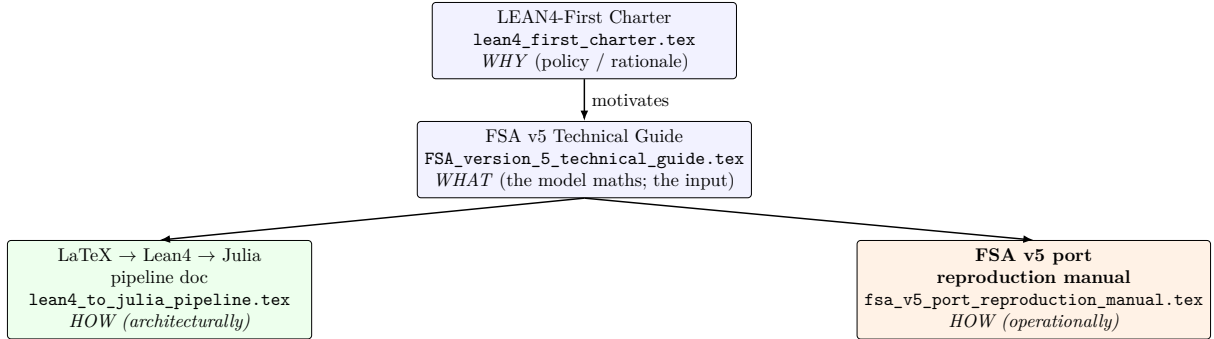


Figure 1: Where this manual fits in the doc set. The two *HOW* docs are complementary: the pipeline doc explains what the components are and why they exist; this manual explains how to build them step by step.

1.4 The seven port phases

Inside the manual, the work is organised into seven phases (Figure 2). Phase 0 is the inputs the reader starts with; Phases 1 and 2 run in parallel and both feed Phase 3; Phases 4–6 are sequential. Each phase has a verification gate that must be green before the next phase begins.

1.5 How to read this manual

Read in order. Treat each phase’s verification gate as a stop-and-check point: only proceed to the next phase when the current gate is green. §10 consolidates all seven gates as a final checklist; §11 lists known gotchas the reader will likely hit.

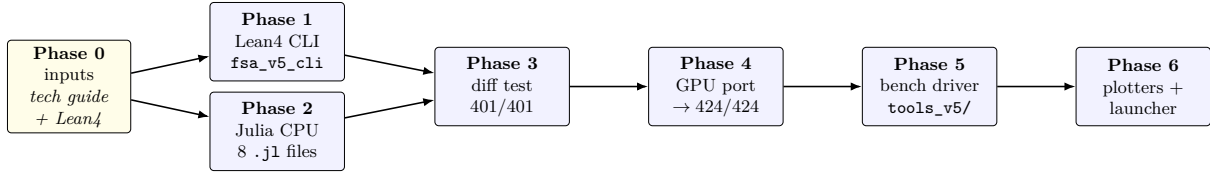


Figure 2: Seven-phase port flow. Phase 0 is the on-disk inputs the reader starts with. Phases 1 and 2 are independent and run in parallel; both feed Phase 3 (the differential test, which gates correctness). Phase 4–6 are sequential. Each phase has a verification gate listed in §10.

2 Prerequisites

- **Hardware.** NVIDIA GPU with CUDA support. The project is tuned for RTX 5090; smaller GPUs (e.g. RTX 3090, A100) will work but at reduced throughput. The reference T=7 run used ~ 5.4 GB peak VRAM.
- **OS.** Linux. Tested on Ubuntu.
- **Toolchain.**
 - Julia ≥ 1.11 , with the project’s manifest resolved (`julia -project=. -e 'using Pkg; Pkg.instantiate()'`).
 - Lean4 with `lake` on the PATH.
 - CUDA driver matching the project’s `CUDA.jl` pin (driver $\geq 590.48.01$ for the RTX 5090’s consumer Blackwell saturated config).
 - The project’s conda env `comfyenv` (per the repo’s `CLAUDE.md`).
- **Repo state.** Clean checkout of `python-smc2-filtering-control`. No `git stash`-ed work in `version_1_5_Julia/` or `version_1_5_LEAN/`.
- **Reading.** Skim sections 1–7 of the FSA v5 technical guide for the model maths. The manual cites specific subsections as needed during the port; don’t memorise it.

3 Phase 0 — Inputs

The reader starts with three on-disk artefacts. Confirm each is present before beginning Phase 1.

Artefact	Path (under repo root)	Role
FSA v5 technical guide	<code>version_1_5_LEAN/LaTeX_docs/FSA_version_5_technical_guide.tex</code>	Math source-of-truth. Every drift / diffusion / observation equation is here.
Lean4 model	<code>version_1_5_LEAN/Fsa/V5/</code>	Six files: <code>Types</code> , <code>Drift</code> , <code>Plant</code> , <code>Obs</code> , <code>Cost</code> , <code>Schedule</code> . Already transcribed from the tech guide; the Julia port mirrors these line-by-line.
v1.5 Julia layout	<code>version_1_5_Julia/models/fsa_high_res/</code>	The pattern to mirror: 7 <code>.jl</code> files, 1 bench-glue file, 2 GPU files. v5 maintains the same shape.

4 Phase 1 — Build the v5 Lean4 CLI

Goal. Produce a native binary `fsa_v5_cli` that the differential test in Phase 3 calls. The binary speaks JSON on stdin/stdout, one request per line.

Inputs. The six `.lean` files in `Fsa/V5/` (complete, verified to compile) and the existing `Main.lean` for v1.5 as the dispatch-pattern reference.

Files to create / modify.

File	New / modified	Purpose
<code>version_1_5_LEAN/Fsa/V5.lean</code>	new, ~ 6 lines	umbrella module: <code>import Fsa.V5</code> pulls all six submodules
<code>version_1_5_LEAN/Main_v5.lean</code>	new, ~ 250 lines	JSON-line dispatcher; one handler per public function
<code>version_1_5_LEAN/lakefile.lean</code>	+5 lines	second <code>lean_exe</code> target named <code>fsa_v5_cli</code>

JSON wire conventions. The dispatcher serialises four record types as JSON:

Lean type	JSON shape
<code>State6D</code>	array of 6 floats: <code>[B, S, F, A, K_FB, K_FS]</code>
<code>BimodalPhi</code>	object <code>{"Phi_B": x, "Phi_S": y}</code>
<code>Params</code>	object with 28 named keys (matches the field order in <code>Types.lean</code>)
<code>ObsParams</code>	object with 22 named keys
<code>Float</code>	decimal; $\pm\text{Inf}$ / <code>NaN</code> use the literal extended-JSON tokens (Julia's <code>JSON3</code> accepts these via <code>allow_inf=true</code>)

Float printing uses Lean's default `toString` which emits 6 significant digits. This is the wire-format precision floor; see §11.

Dispatch tags exposed (15). `drift`, `diffusion`, `emStep`, `hrMean`, `sleepProb`, `stressMean`, `stepsLogMean`, `volumeLoadMean`, `muBar`, `findASep`, `aSepGrid`, `scheduleFromTheta`, `designMatrix`, `cPhi`, `sigmoid`.

Two surfaces are deliberately *not* exposed:

- `applyPrior`. Lives in v1.5's `Fsa/V15/Estimation.lean`; not duplicated for v5. The prior transform is identical across versions, so the v5 diff test reuses the v1.5 binary if it ever needs to verify `applyPrior`.
- `obsLogWeight`. The v5 likelihood is a sum of five per-channel Gaussian / Bernoulli log-likelihoods. The per-channel *means* (`hrMean`, `stressMean`, etc.) are exposed individually; the sum is composed Julia-side. Keeps the Lean surface minimal and avoids redundant test paths.

Procedure.

1. Create `Fsa/V5.lean` with one `import` per submodule.
2. Create `Main_v5.lean` mirroring `Main.lean`'s structure: `handleRequest` dispatches on the JSON `"fn"` field; per-tag handlers parse the typed record from JSON, call the corresponding `Fsa.V5.*` function, and serialise the result.
3. In `lakefile.lean`, add a second `lean_exe`:

```

1 @[default_target]
2 lean_exe fsa_v5_cli where
3   root := 'Main_v5

```

```

1 cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_LEAN
2 lake build
3 # Both binaries should appear:
4 ls .lake/build/bin/fsa_v15_cli .lake/build/bin/fsa_v5_cli
5
6 echo '{"fn":"sigmoid","x":0.0}' | .lake/build/bin/fsa_v5_cli
7 # Expect: {"x":0.500000}
8
9 echo '{"fn":"cPhi","phi_default":1.0,"phi_max":3.0}' | .lake/build/bin/
   fsa_v5_cli
10 # Expect: {"x":-0.693147}      (i.e.\ -log(2))

```

If both invocations return the expected values, Phase 1 is green.

5 Phase 2 — Write the Julia CPU port

Goal. Create eight `.jl` files under `version_1_5_Julia/models/fsa_v5/` that transcribe each Lean4 module into a Julia equivalent. Mirror the v1.5 layout at `version_1_5_Julia/models/fsa_high_res/` exactly: each Julia file has a one-to-one Lean source.

Files to create.

File	Lean source	What it provides
<code>simulation_v5.jl</code>	<code>Types.lean</code> , tech guide §6	constants <code>BINS_PER_DAY=96</code> , <code>DT_BIN_DAYS=1/96</code> ; <code>TRUTH_PARAMS_V5</code> , <code>DEFAULT_OBS_PARAMS_V5</code> , <code>DEFAULT_INIT</code> , <code>TRAINED_ATHLETE_INIT</code> , <code>FROZEN_PARAMS_V5</code> , <code>PARAM_KEYS_V5</code> , <code>OBS_PARAM_KEYS_V5</code>
<code>_dynamics_v5.jl</code>	<code>Drift.lean</code>	<code>drift_v5</code> , <code>diffusion_v5</code> , <code>em_step_v5</code> . Hill exponent <code>n_dec</code> hardcoded to 4 (frozen per tech guide §7.2).
<code>_plant_v5.jl</code>	<code>Plant.lean</code>	<code>PlantState6D</code> , <code>plant_step_v5</code> , <code>plant_rollout_v5</code> . NB rollout return shape is bench-consumed (flat <code>NamedTuple</code>), not Lean-counterpart.
<code>obs_v5.jl</code>	<code>Obs.lean</code>	<code>hr_mean</code> , <code>sleep_prob</code> , <code>stress_mean</code> , <code>steps_log_mean</code> , <code>volume_load_mean</code>
<code>estimation_v5.jl</code>	<code>Obs.lean</code> + tech guide §3	<code>obs_log_weight_v5</code> (5-channel sum, per-channel gates), <code>propagate_v5</code> , <code>PARAM_NAMES_V5</code> (37 entries), <code>PARAM_PRIOR_CONFIG_V5</code>
<code>cost_v5.jl</code>	<code>Cost.lean</code>	<code>mu_bar</code> , <code>find_a_sep</code> ($\pm\text{Inf}$ sentinels), <code>a_sep_grid</code> (per-particle $\times$ per-bin matrix)
<code>schedule_v5.jl</code>	<code>Schedule.lean</code>	<code>sigmoid</code> , <code>c_phi</code> , <code>schedule_from_theta</code> , <code>design_matrix</code>
<code>FSAv5.jl</code>	aggregator	includes the seven submodules in dependency order; re-exports 39 public symbols (47 after Phase 4 adds the GPU files)

Lean $\rightarrow$ Julia idiom conversions. The transcription is mechanical but a handful of Lean idioms map specifically to Julia:

Lean	Julia
<code>Id.run</code> do mutable loop	ordinary <code>for</code> loop with mutable accumulator
<code>Float.pow</code> ($\max x \ 0.0$) <code>n</code>	<code>max(x, 0.0)^n</code>
<code>1.0 / 0.0</code> ($+\infty$ sentinel)	<code>Inf</code> (native, no extra plumbing)
<code>-1.0 / 0.0</code> ($-\infty$ sentinel)	<code>-Inf</code>
<code>Array</code> (<code>Array Float</code>)	<code>Matrix{Float64}</code> (rectangular) or <code>Vector{Vector{Float64}}</code> (irregular)
state record	<code>StaticArrays.SVector{6, Float64}</code> for zero-allocation hot paths
<code>Params</code> record	<code>Dict{Symbol, Float64}</code> (with <code>NamedTuple</code> -converter helper for <code>ForwardDiff</code> compatibility)

Boundary handling — a deliberate v5 deviation from v1.5. v1.5’s `em_step_substepped` reflects B at the $[0, 1]$ boundary and `abs`-folds F, A . v5 instead *clamps* B, S to $[\varepsilon, 1-\varepsilon]$ and floors F, A, K_{FB}, K_{FS} at 0, per tech guide §6.4. The Lean `Plant.lean` encodes the clamp-and-floor convention; the Julia transcription must match.

```

1 cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
2 julia --project=. -e '
3 include("models/fsa_v5/FSAv5.jl"); using .FSAv5
4 @assert sigmoid(0.0) == 0.5
5 @assert isapprox(c_phi(1.0, 3.0), -log(2.0))
6 @assert isapprox(drift_v5(zeros(6), TRUTH_PARAMS_V5, (0.0, 0.0))[5],
7                     TRUTH_PARAMS_V5[:KFB_0] / TRUTH_PARAMS_V5[:tau_K])
8 @assert isinf(find_a_sep((0.30, 0.30), TRUTH_PARAMS_V5))
9 @assert find_a_sep((0.30, 0.30), TRUTH_PARAMS_V5) < 0
10 # tech guide section 4.5: balanced moderate is healthy mono-stable -> -Inf
11 println("Phase 2 smoke-load: PASS")
12 '
```

If the script prints `PASS`, Phase 2 is green.

6 Phase 3 — Differential test

6.1 Background: what is a differential test, and why use one here?

A **differential test** is a test that checks two independent implementations of the same mathematical function and asserts they produce the same answer on every input. Think of it as spell-checking a translation by feeding the same paragraph into two dictionaries: if their definitions disagree on any word, you know that word was mistranslated.

Why a differential test is the right tool here. The work in Phases 1 and 2 is, at its core, *transcription*. The maths already exists (in the FSA v5 technical guide and the Lean4 source); the Julia code is supposed to encode the same arithmetic in a different programming language. The bugs that show up in this kind of work are typos, sign flips, swapped arguments, off-by-one indexing — not algorithmic errors. These bugs are very hard to spot by reading the code, and very easy to expose by feeding the same numerical input to both implementations and watching for a disagreement on the output. A differential test does exactly that, mechanically, on hundreds of inputs in a few seconds.

Who plays which role? The Lean4 binary `fsa_v5_cli` (Phase 1’s deliverable) is the *reference*. It is small, type-checked, and was reviewed line-by-line against the technical guide — it is treated as the ground truth. The Julia code from Phase 2 is the *implementation under test*; the diff test asks “does Julia produce the same number Lean did?” and flags any disagreement.

How the two sides talk to each other. Lean4 compiles to a native binary that reads one JSON request per line on stdin and writes one JSON response per line on stdout. The Julia test driver:

1. launches the binary as a long-lived subprocess (*one* startup cost per test run, not per assertion);
2. for each test case, builds a JSON object describing the function call (e.g. `{"fn": "drift", "state": [...], "phi": {...}, "params": {...}}`), writes it on stdin, and reads the JSON response on stdout;
3. runs the same call locally in Julia;
4. asserts `|Julia - Lean4| < tol` component-by-component (or `isapprox`-style for arrays).

The subprocess + JSON-line protocol is just a robust way of calling a function across a programming-language boundary (Lean and Julia don’t share a runtime). It is mechanically simple; the same pattern works for any pair of compiled-to-native languages.

The pre-drawn-noise trick (and why it matters). Some functions in the model involve random numbers — in particular, the Euler–Maruyama step in `em_step_v5` adds a Gaussian noise increment to advance the SDE by one bin. If Julia and Lean each drew their own noise from their respective random-number generators, they would draw *different* numbers (the two languages have different RNGs and even different default seeds), so the outputs would always differ — not because the maths disagrees but because the inputs were different. That is a false fail.

The fix is the **pre-drawn noise** convention: the test driver draws the noise vector *once* on the Julia side, then passes that exact noise array to *both* the Julia call and the Lean call. Both implementations now run their EM step on *identical* noise. Any output disagreement is purely due to the deterministic arithmetic (sign error, wrong coefficient, swapped state component, etc.) — which is what we actually want to detect.

Why a tolerance, not strict equality. Floating-point arithmetic is not exact. Two valid implementations of the same formula can produce results that differ in the last few bits because of how the rounding accumulates through the operations (IEEE-754 round-off). Insisting on `==` would fail correctly-written code. The diff test instead asks “do the two results agree to *n* digits?”. The threshold is the **tolerance**.

The manual uses two tolerances. The bulk of the tests use 10^{-6} (roughly six matching digits): tight enough to catch any real bug, loose enough to absorb the round-off of single-arithmetic-operation disagreement. The integrated EM step uses a looser 10^{-4} because it composes 4 substeps plus a Wiener increment, so round-off accumulates more. The exact tolerance for each function is in the table below; the rationale per row is the only piece a fresh reader needs to internalise.

What “green” means. A green test run means: every Julia function in Phase 2, on every random input the test threw at it, produced a number indistinguishable from Lean’s at the prescribed tolerance. The reverse is also true: *any* red assertion points at a specific function whose Julia code disagrees with the Lean reference, and the assertion message tells you exactly which input triggered the disagreement and by how much. That makes debugging a port mechanical: you do not have to read the code; the test tells you which line to fix.

6.2 Implementation in this manual

Goal. Verify every Julia function from Phase 2 against its Lean4 reference at machine precision. The differential test spawns the `fsa_v5_cli` binary as a long-lived subprocess, sends each function’s inputs as JSON on stdin, reads the Lean4 result on stdout, runs the same call locally in Julia, and asserts $|\text{Julia} - \text{Lean4}| < \text{tol}$.

File to create. `version_1_5_Julia/diff_test/test_lean_diff_v5.jl` (~ 440 lines). Mirrors `test_lean_diff_v15.jl`’s structure (long-lived subprocess client, `round_trip` helper, pre-drawn noise).

Pre-drawn noise convention. For `em_step_v5` (the only function that integrates noise) both sides receive the same 6-vector standard-normal noise as input rather than each drawing its own. RNG-divergence between Julia and Lean is excluded from the comparison; the test is purely about the deterministic numerical agreement of the math.

Tolerance ladder.

Class	Tolerance	Functions
single-step deterministic	10^{-6} atol/rtol	<code>drift_v5</code> , <code>diffusion_v5</code> , <code>plant_step</code> , all 5 channel means, <code>mu_bar</code> , <code>find</code>
integrated EM step	10^{-4} atol	<code>em_step_v5</code> (4-substep EM + Wiener increment)
schedule outputs at magnitude ~ 3	10^{-5} rtol	<code>schedule_from_theta</code> (Lean’s 6-sig-digit <code>toString</code> is the wire-format pre

```
1 cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_LEAN && lake
  build
2 cd ../version_1_5_Julia
3 julia --project=. diff_test/test_lean_diff_v15.jl
4 # Expect: Test Summary: FSA v1.5 LEAN4 diff | Pass 121 | Total 121
5 julia --project=. diff_test/test_lean_diff_v5.jl
6 # Expect: Test Summary: FSA v5 LEAN4 diff | Pass 401 | Total 401
```

The v1.5 test must still pass — this confirms no regression on the existing surface. The v5 test reports 401 / 401 here; Phase 4 extends it to 424 / 424.

7 Phase 4 — GPU port (SOFT cost only)

Goal. Produce two GPU files that run the v5 SDE math on-device: `gpu_pf_v5.jl` (the inner-PF kernel for the filter) and `gpu_control_v5.jl` (the cost kernel for the controller). Each file mirrors its v1.5 counterpart’s plumbing verbatim and only swaps the math block.

Architectural rule (cite the LEAN4-First Charter).

Copy v1.5’s plumbing verbatim — kernel structure, RNG-per-thread layout, log-likelihood reduction, NaN guards, framework hooks. Only swap the per-particle (or per-trial) math block with the v5 versions transcribed from the diff-tested CPU functions. Do NOT rewrite the plumbing; that is the AI-fuckup-prone surface.

Dimension bumps (the only changes inside the kernel plumbing).

- Particle state dim: $3 \rightarrow 6$ (allocations, indexing, packing).
- Obs dim: $3 \rightarrow 5$ Gaussian + Bernoulli, with explicit per-bin gates as `Float32` masks (multiply each channel’s log-likelihood by its gate).
- Params per chain: $14 \rightarrow 15$ estimated dynamics + 22 estimated obs-channel + 14 frozen.
- Boundary: reflect $\rightarrow$ clamp/floor (matches Phase 2).

SOFT cost only. The v5 cost kernel implements

$$J(\theta) = \lambda_\Phi \int_0^T (\Phi_B^2 + \Phi_S^2) dt - \int_0^T A dt + \lambda_F \int_0^T \max(F - F_{\max}, 0)^2 dt + \lambda_{\text{chance}} \int_0^T \sigma\left(\beta \cdot \frac{A_{\text{thr}} - A}{\text{scale}}\right) dt.$$

The chance surrogate uses a constant A_{thr} (default 0.05) rather than the per-bin $A_{\text{sep}}(\Phi_t)$ from `find_a_sep`. The full per-bin form would require either a CPU-side precompute or a separate GPU prep kernel ($\sim 286\text{M}$ `mu_bar` evaluations per call at production sizes); deferred. The HARD chance-constraint variant (with indicator instead of sigmoid) is deliberately out of scope.

Single-thread debug entry points. Each kernel exposes a 1-thread debug function that the diff test invokes for `ndrange = 1`:

- `gpu_propagate_one!` runs the PF kernel for one (*chain, particle, bin*) and returns the next 6D state and the accumulated log-weight.
- `gpu_cost_one!` runs the controller cost kernel for one (*chain, trial*) and returns the scalar cost.

These let the diff test compare the GPU per-thread math against the diff-tested CPU functions (`em_step_v5`, `obs_log_weight_v5`, the inline cost replicate) at fp32 precision ($\sim 10^{-3}$ absolute). Closes the GPU-coverage gap documented in the pipeline doc §8.

HMC clarification — both filter and controller HMC live in the framework, not in the model. The v5 model exposes two closures:

- `gpu_log_density_v5(target, U, grid_obs, key)` — the filter’s $\log p(y | \theta)$ closure.
- `make_log_density_fn_v5(ctrl_target)` — the controller’s $-\text{cost}(\theta)$ closure.

`SMC2FC_functional` consumes these via `parallel_hmc_one_move_generic! + chees_pick_L_generic` (filter) and `run_tempered_smc_gpu` (controller). **No new HMC code is needed in the model directory.** The legacy model-specific `parallel_hmc_one_move` in v1.5’s `gpu_pf.jl` is dead code.

Files to create / modify.

File	Lines	Purpose
<code>models/fsa_v5/gpu_pf_v5.jl</code>	~ 430	<code>propagate_segment</code> kernel + log-lik accumulator + <code>FSAv5GPUPTarget</code> + <code>gpu_log_weight_v5</code>
<code>models/fsa_v5/gpu_control_v5.jl</code>	~ 330	v5 cost kernel + <code>FSAv5ControlGPUPTarget</code> + <code>gpu_cost_log_density_batch</code>
<code>models/fsa_v5/FSAv5.jl</code>	+12 lines	<code>include</code> + re-export the two GPU files (47 exports total)
<code>diff_test/test_lean_diff_v5.jl</code>	+ ~ 200 lines	two new testsets exercising the single-thread debug entries

```

1 cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
2 julia --project=. diff_test/test_lean_diff_v5.jl
3 # Expect: Test Summary: FSA v5 LEAN4 diff | Pass 424 | Total 424

```

That is, 401 from Phase 3 plus 23 new GPU parity asserts (21 state components + 1 log-weight + 2 cost = 24 minus 1 for the cost trial running 2-pass = 23). At fp32 vs fp64 precision (tolerance 10^{-3} absolute).

8 Phase 5 — Bench driver

Goal. Produce a runnable closed-loop SMC²-MPC bench: a top-level driver script plus five bench-helper modules and one model-specific glue file.

Honest flag about “model-agnostic”. The v1.5 `tools/bench/` subdir is generic in *flow* (foldl over strides, tempered SMC² ladder, ChEES-HMC, RBF schedule decoder) but not in *symbols*. The bench files hardcode `FSAGPUTarget`, `gpu_log_density`, `posterior_mean_v15`, `PARAM_NAMES`, `DEFAULT_PARAMS`, `PINNED_PARAMS`, the 3-channel obs schema, and the `SVector{3}` state extraction. The v5 port copies the structure and substitutes the v5 names / shapes in place. It is *not* a verbatim copy.

Files to create.

File	Lines	Source / change
<code>tools_v5/bench_smc_full_mpc_fsa_v5_gpu.jl</code>	~ 250	top-level driver; mirrors <code>bench_smc_full_mpc_fsa_gpu.jl</code> with v5 in
<code>tools_v5/bench/bench_args.jl</code>	~ 270	verbatim copy of v1.5 + <code>step-minutes</code> default 60 → 15
<code>tools_v5/bench/bench_filter.jl</code>	~ 230	sed-renamed: <code>FSAGPUTarget</code> → <code>FSAv5GPUPUTarget</code> ; <code>gpu_log_density</code> →
<code>tools_v5/bench/bench_controller.jl</code>	~ 120	rewritten: 6D init, params Dict, θ -dim = $2 \cdot n_{\text{anchors}}$, decoder product
<code>tools_v5/bench/bench_loop.jl</code>	~ 210	rewritten: 13-field obs accumulator (5 obs + 5 gates + Φ_B + Φ_S + Φ_C)
<code>tools_v5/bench/bench_postproc.jl</code>	~ 280	sed-renamed; data.jld2 schema bumped to 1.0-v5 with <code>state_names</code>
<code>models/fsa_v5/bench_glue_v5.jl</code>	~ 110	<code>posterior_mean_v5</code> (50-key Dict: 37 estimated + 13 frozen), window

Plant-rollout shape change in `_plant_v5.jl` (ripple edit). The Phase 2 `plant_rollout_v5` returned `(trajectory, obs_seq)`. Phase 5 changes it to a flat `NamedTuple` matching the bench’s `accumulate_obs_history_v5` consumer (separate `obs_HR` / `obs_S` / `obs_steps` / `obs_VL` / `obs_sleep` / `Phi_B` / `Phi_S` / `C` / `final_state` / `trajectory` fields). The function is not diff-tested against Lean (no Lean counterpart), so the shape change does not invalidate any test.

```

1 cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
2 FSA_V5_DRY_RUN=1 julia --project=. tools_v5/bench_smc_full_mpc_fsa_v5_gpu.jl --
  T-days 1
3 # Expected: two log lines and exit 0:
4 # [ Info: loading v5 model + framework (FSA_STEP_MINUTES=15)...
5 # [ Info: FSA_V5_DRY_RUN=1 -- driver loaded but main() skipped

```

The `FSA_V5_DRY_RUN` guard at the bottom of the driver makes the include-and-link smoke test possible without running the bench.

9 Phase 6 — Plotters, production gating, and launchers

Three plotters (state traces, parameter traces, observation channels), one production gating-logic addition to the bench, and two launcher shell scripts.

Files to create.

File	Lines	Purpose
tools_v5/plot_state_traces_v5.jl	~ 120	8-panel state plot: 6 state components ($B, S, F, A, K_{FB}, K_{FS}$) + 2 daily
tools_v5/plot_param_traces_v5.jl	~ 95	5 × 8 grid of per-parameter posterior trace + 5/95% quantile band + t
tools_v5/plot_obs_channels_v5.jl	~ 190	6-panel obs plot: HR / Stress / Steps / VL / Sleep + circadian, with c
tools_v5/launchers/run_julia_v5_T42d.sh	~ 120	T=42 default launcher; <code>nvidia-smi</code> 1 Hz sampler + bench + 3 plotters
tools_v5/launchers/run_julia_v5_T7d.sh	~ 120	smoke-test variant; T=7 default

Production sleep / wake gating (tech guide §3.2). Modify `bench_loop.jl::accumulate_obs_history_v` to compute per-bin gates from the bin’s hour-of-day rather than emitting all-ones:

Channel	Gate rule
HR	sleep window: $h \in [22, 24) \cup [0, 6)$
Stress	wake (complement of sleep)
Steps	wake (complement of sleep)
VolumeLoad	one bin per day at 18:00 (= bin 72 in 0-indexed bin-within-day at <code>BINS_PER_DAY=96</code>)
Sleep label	every bin (always 1.0)

Verification arithmetic at `BINS_PER_DAY=96`:

Quantity	Expected value
<code>sum(gate_HR)</code>	32 (= 8 h × 4 bins/h)
<code>sum(gate_stress)</code>	64 (= 16 h × 4 bins/h)
<code>sum(gate_VL)</code>	1 (single bin at 18:00)
<code>sum(gate_sleep)</code>	96 (every bin)

bench_postproc.jl additions. Save the 5 obs vectors, 5 gate masks, and circadian $C(t)$ to `data.jld2` so the obs-channel plotter has all required inputs. The fields are documented in §A and consumed by `plot_obs_channels_v5.jl`.

```

1 cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
2 chmod +x tools_v5/launchers/run_julia_v5_T7d.sh
3 ./tools_v5/launchers/run_julia_v5_T7d.sh 7 \
4   --N-smc 32 --K-per-chain 200 --ctrl-n-smc 256 \
5   --ctrl-num-mcmc 8 --ctrl-chees-max 256 \
6   --ctrl-n-anchors 8 --ctrl-max-levels 25

```

This is the Phase 6 verification gate and it doubles as the end-to-end manual gate. Expected wall on RTX 5090: ~ 6 minutes. The output dir `compare_v15_julia_vs_python/example_run/julia_v5_T7d_<DATE>/T7d_seed42/` should contain:

```

1 bench.log
2 controller_diagnostics.csv
3 data.jld2
4 experiment_run.md
5 manifest.json
6 nvidia_smi.csv
7 per_stride.csv
8 v5_T7d_obs_channels.png
9 v5_T7d_param_traces.png
10 v5_T7d_traces.png

```

10 Verification gates — final checklist

After completing all six phases, run these gates in order. Each must be green before claiming the port is reproduced.

1. **Lean binaries build.**

```
cd version_1_5_LEAN && lake build
```

Pass: both `fsa_v15_cli` and `fsa_v5_cli` appear under `.lake/build/bin/`.

2. **v1.5 diff test still passes.**

```
julia -project=. diff_test/test_lean_diff_v15.jl
```

Pass: Pass 121 | Total 121.

3. **v5 diff test passes.**

```
julia -project=. diff_test/test_lean_diff_v5.jl
```

Pass: Pass 424 | Total 424 (8 model + 5 obs-channel + 5 cost/schedule + 23 GPU parity asserts; see §C).

4. **Driver dry-run loads cleanly.**

```
FSA_V5_DRY_RUN=1 julia -project=. tools_v5/bench_smc_full_mpc_fsa_v5_gpu.jl  
-T-days 1
```

Pass: prints the load banner and the dry-run skip; exit 0.

5. **End-to-end T=7 smoke run.**

```
./tools_v5/launchers/run_julia_v5_T7d.sh 7 --N-smc 32 ...
```

Pass: exit 0; output dir contains 10 artefacts listed in §9; wall time ~ 6 min on RTX 5090.

6. **Reference-run comparison.** The verified reference run from 2026-05-09T22:01:01 had wall = 362 s, 13 strides, 37 estimated parameters, 13 frozen, and recovered final state (B=0.0922, S=0.1098, F=0.3471, A=0.0409, K_FB=0.0488, K_FS=0.0652). Pass: the reproducer’s run matches this within stochastic variation — order of magnitude on each component; qualitative trajectory shape; same artefact set produced.

11 Troubleshooting and known gotchas

Lean’s `Float.toString` floor at 6 significant digits. Manifests as `schedule_from_theta` test failures at 10^{-6} atol when the output magnitude is ~ 3 ($\Phi_{\max} \cdot \text{sigmoid}$). Fix: use `rtol=1e-5` for schedule outputs. Documented in `test_lean_diff_v5.jl`.

GPU plumbing reuse. Don’t refactor the kernel / framework glue when porting v5. Verbatim copy from v1.5 with math-block swap is the rule. Refactoring there is the AI-fuckup-prone surface.

fp32 inner loops, fp64 outer state. Project convention (see `CLAUDE.md` GPU-dtype section). Do not annotate `Float64` inside SDE inner loops on the GPU; doing so requests fp64 throughout and tanks RTX 5090 throughput by $\sim 64\times$.

FSA_STEP_MINUTES env var must be set before model import. The bench driver sets `ENV["FSA_STEP_MINUTES"] = ...` before `include("FSAv5.jl")`. Reorder this and `BINS_PER_DAY` resolves to the wrong value because `simulation_v5.jl` reads the env var at module-load time.

FSA_V5_DRY_RUN=1 for dry-run smoke testing. Loads the driver and all its includes but skips `main()`. Useful for catching parse / unresolved-symbol errors without running the bench.

ot-max-weight = 0.0 default. Setting this > 0 enables the framework's OT rescue with a documented $\sim 12\times$ wall-time penalty (per-chain Julia loop with PCIe round-trips). v5's strong direct-Gaussian + 5-channel obs model does not need OT rescue at the saturated config.

HARD chance-constraint deliberately out of scope. Only SOFT (Lagrangian-relaxed with constant A_{thr} , default 0.05) is implemented. The HARD form (using `find_a_sep`-derived per-bin A_{sep} as an indicator) requires either CPU-side precompute of A_{sep} per (chain, bin) or a separate GPU prep kernel. Deferred.

@match site numbering. v1.5 has 3 @match sites (`paramsV15ToV1`, `reflect_unit`, `apply_prior`). v5 has zero — no basis adapter (v5 uses its own basis directly), no reflection (uses clamp/floor instead), no separate prior transform exposed at the JSON layer. Don't look for v5 @match sites.

If a phase gate fails. Don't proceed to the next phase. Almost every failure mode reduces to either (a) a wrong field order in the JSON wire (Lean side) or the param Dict (Julia side), or (b) an incorrect Lean $\rightarrow$ Julia idiom translation in the math block. The diff test is precise about which function disagrees; re-read the corresponding `Fsa/V5/<file>.lean` and the Julia transcription side by side.

A Complete file tree (post-port)

After all six phases:

```

1 version_1_5_LEAN/
2   Fsa/V5/                                (Phase 0 input; six files)
3   Fsa/V5.lean                            (Phase 1 new)
4   Main_v5.lean                          (Phase 1 new)
5   lakefile.lean                        (Phase 1 +5 lines)
6   LaTeX_docs/
7     fsa_v5_port_reproduction_manual.tex (this doc)
8
9 version_1_5_Julia/
10  models/fsa_v5/                         (Phase 2-4-5 new directory)
11    FSAv5.jl                             (aggregator)
12    simulation_v5.jl
13    _dynamics_v5.jl
14    _plant_v5.jl
15    obs_v5.jl
16    estimation_v5.jl
17    cost_v5.jl
18    schedule_v5.jl
19    gpu_pf_v5.jl                         (Phase 4)
20    gpu_control_v5.jl                   (Phase 4)
21    bench_glue_v5.jl                   (Phase 5)
22    PORT_LOG.md                        (chronological record)
23
24  diff_test/
25    test_lean_diff_v5.jl                (Phase 3 + extended in Phase 4)

```

```

26
27 tools_v5/                                (Phase 5 + 6 new directory)
28   bench_smc_full_mpc_fsa_v5_gpu.jl
29   bench/
30     bench_args.jl
31     bench_filter.jl
32     bench_controller.jl
33     bench_loop.jl
34     bench_postproc.jl
35   plot_state_traces_v5.jl                (Phase 6)
36   plot_param_traces_v5.jl                (Phase 6)
37   plot_obs_channels_v5.jl                (Phase 6)
38   launchers/
39     run_julia_v5_T42d.sh
40     run_julia_v5_T7d.sh

```

data.jld2 schema produced by Phase 6's bench:

```

1 trajectory_mpc                # (n_bins, 6)   Float32
2 trajectory_baseline           # (n_bins, 6)   Float32
3 Phi_B_per_bin_mpc             # (n_bins,)    Float32
4 Phi_S_per_bin_mpc             # (n_bins,)    Float32
5 Phi_B_per_bin_baseline        # (n_bins,)    Float32
6 Phi_S_per_bin_baseline        # (n_bins,)    Float32
7 daily_phi_per_stride          # (n_strides,) Float32
8 state_names                   # ["B","S","F","A","K_FB","K_FS"]
9 posterior_particles           # (n_strides, n_smc, 37) Float64 (constrained)
10 posterior_window_mask         # (n_strides,) Bool
11 param_names                   # 37 Strings
12 truth_params_dict             # Dict{String, Float64}, 50 keys
13 obs_HR, obs_S, obs_steps      # (n_bins,)    Float32
14 obs_VL, obs_sleep             # (n_bins,)    Float32
15 gate_HR, gate_stress          # (n_bins,)    Float32 (0 or 1)
16 gate_steps, gate_VL          # (n_bins,)    Float32
17 gate_sleep                    # (n_bins,)    Float32
18 circadian_C                   # (n_bins,)    Float32
19 BINS_PER_DAY, STRIDE_BINS     # Int
20 WINDOW_BINS                   # Int
21 dt_days, F_max, wall_seconds  # Float64
22 seed                          # Int

```

B Command index

Every shell / Julia / lake command in the manual, in execution order:

```

1 # Phase 1 verification
2 cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_LEAN
3 lake build
4 echo '{"fn":"sigmoid","x":0.0}' | .lake/build/bin/fsa_v5_cli
5 echo '{"fn":"cPhi","phi_default":1.0,"phi_max":3.0}' | .lake/build/bin/
   fsa_v5_cli
6
7 # Phase 2 verification
8 cd ../version_1_5_Julia
9 julia --project=. -e '
10 include("models/fsa_v5/FSAv5.jl"); using .FSAv5
11 @assert sigmoid(0.0) == 0.5
12 @assert isapprox(c_phi(1.0, 3.0), -log(2.0))
13 @assert isapprox(drift_v5(zeros(6), TRUTH_PARAMS_V5, (0.0, 0.0))[5],
14                   TRUTH_PARAMS_V5[:KFB_0] / TRUTH_PARAMS_V5[:tau_K])
15 @assert isinf(find_a_sep((0.30, 0.30), TRUTH_PARAMS_V5))
16 println("PASS")
17 '

```

```

18
19 # Phase 3 verification
20 julia --project=. diff_test/test_lean_diff_v15.jl # 121/121
21 julia --project=. diff_test/test_lean_diff_v5.jl # 401/401
22
23 # Phase 4 verification (extended diff test)
24 julia --project=. diff_test/test_lean_diff_v5.jl # 424/424
25
26 # Phase 5 verification (dry-run smoke)
27 FSA_V5_DRY_RUN=1 julia --project=. tools_v5/bench_smc_full_mpc_fsa_v5_gpu.jl --
    T-days 1
28
29 # Phase 6 verification (end-to-end T=7 smoke)
30 chmod +x tools_v5/launchers/run_julia_v5_T7d.sh
31 ./tools_v5/launchers/run_julia_v5_T7d.sh 7 \
32     --N-smc 32 --K-per-chain 200 --ctrl-n-smc 256 \
33     --ctrl-num-mcmc 8 --ctrl-chees-max 256 \
34     --ctrl-n-anchors 8 --ctrl-max-levels 25

```

C Differential-test coverage table

Testset	Asserts	Tolerance
sigmoid	12	10^{-6}
c_phi	4	10^{-6}
drift_v5	30	10^{-6}
diffusion_v5	30	10^{-6}
em_step_v5	30	10^{-4} (integrated)
hr_mean	5	10^{-6}
sleep_prob	5	10^{-6}
stress_mean	5	10^{-6}
steps_log_mean	5	10^{-6}
volume_load_mean	5	10^{-6}
mu_bar	5	10^{-6}
find_a_sep	7	10^{-6} + sentinel checks
a_sep_grid	15	10^{-6}
schedule_from_theta	237	10^{-5}
Phase 3 subtotal	401	
gpu_pf_v5 single-thread parity	21	10^{-3} (fp32 vs fp64)
gpu_control_v5 single-thread parity	2	10^{-3}
Phase 4 subtotal	23	
Total	424	

References

- [1] *A LEAN4-First Charter for FSA Model Development.*
version_1_5_LEAN/LaTeX_docs/lean4_first_charter.tex. Policy doc: why Lean4 is the formal source of truth and Julia is the differentially-tested artefact.
- [2] *FSA-v5 Technical Implementation Guide.*
version_1_5_LEAN/LaTeX_docs/FSA_version_5_technical_guide.tex. The model maths source-of-truth for Phase 0 inputs.
- [3] *Practical pipeline documentation: LaTeX $\rightarrow$ Lean4 $\rightarrow$ Julia for FSA v1.5.*
version_1_5_LEAN/LaTeX_docs/lean4_to_julia_pipeline.tex. Architecture-level companion to this operational manual.

- [4] *PORT_LOG.md*.
version_1_5_Julia/models/fsa_v5/PORT_LOG.md. Chronological record of WHAT was done; this manual condenses it into HOW.